

Programación funcional y tipos inductivos

Algoritmos y Estructuras de Datos II -
Estructuras de Datos y Algoritmos
Dr. Pablo Ponzio
Universidad Nacional de Río Cuarto
CONICET



El Lenguaje Haskell

- Haskell fue introducido en 1987 con el objetivo de desarrollar un lenguaje funcional moderno
- Vamos a usar el compilador Glasgow Haskell, que se puede obtener en <http://www.haskell.org/ghc/>.

Tipos Básicos de Haskell

Haskell tiene un conjunto rico de tipos básicos de datos

- **Booleanos**, tipos de booleanos con las operaciones lógicas,
- **Int**: Enteros de precisión fija,
- **Char**: 'a', 'b', 'c', etc
- **Integer**: Enteros de precisión variable,
- **Float**: Números reales.

Escribimos: $E :: T$ cuando E es de tipo T

<code>5 :: Integer</code>	
<code>'a' :: Char</code>	<code>[1,2,3] :: [Integer]</code>
<code>inc :: Integer -> Integer</code>	<code>('b',4) :: (Char,Integer)</code>

El Tipo Bool

El tipo `Bool` tiene dos valores `true` y `false`, y las siguientes operaciones:

- `&& :: Bool -> Bool -> Bool`
- `|| :: Bool -> Bool -> Bool`
- `not :: Bool -> Bool`

Cualquier función: `f :: A -> Bool` es llamado predicado, y puede utilizarse con estas operaciones.

`== :: A -> A -> Bool`

La igualdad es la más conocida

El Tipo Char

El tipo `char` contiene los valores `'a'`, `'b'`, `'c'`, ... etc, y las siguientes funciones:

- `ord :: Char -> Int` convierte caracteres a enteros.
- `chr :: Int -> Char` convierte enteros a caracteres.

Los Strings se modelan como una lista de chars.

Tuplas

Usando los tipos básicos podemos construir tuplas y listas.
Dados tipos A y B :

(A, B)

El tipo de pares de A y B

Por ejemplo: $(\text{True}, 1) :: (\text{Bool}, \text{Int})$

$\text{fst} :: (a, b) \rightarrow a$

Devuelve el primer
componente

Operaciones:

$\text{snd} :: (a, b) \rightarrow b$

Devuelve el segundo
componente

Listas

Dado un tipo cualquiera a :

$[a]$ Es el tipo a las listas con elementos de tipo a

Las listas son tipos inductivos, y sus valores se crean usando los siguientes constructores:

- $[]$ es la lista vacía
- $x : xs$ es la lista con x a la cabeza y luego xs a la cola

Por ejemplo:

La lista $[1, 2, 3]$ se construye con: $1 : (2 : (3 : []))$

Operaciones sobre Listas

Algunas funciones útiles sobre listas:

- `head :: [a] -> a`, devuelve la cabeza de la lista.
- `last :: [a] -> a`, devuelve el último elemento.
- `tail :: [a] -> [a]`, devuelve la cola de la lista.
- `length :: [a] -> Int`, retorna la longitud de la lista.
- `++ :: [a] -> [a] -> [a]`, concatena dos listas.

Funciones

Dados dos tipos a y b :

$a \rightarrow b$

Es el tipo de las
funciones de a en b

Por ejemplo:

$\text{Not} : \text{Bool} \rightarrow \text{Bool}$

Función de Bool en
 Bool

Las funciones de alto orden son aquellas que toman como parámetros funciones o retornan funciones

$:: (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow (a \rightarrow c)$

Definir la composición de
funciones

Definición de Funciones

Una función se puede definir por casos:

```
sign :: Int -> Int
sign x | x >= 0 = 1
       | x < 0  = -1
```

También usando recursión y pattern matching:

```
suma :: [Int] -> Int
suma [] = 0
suma (x:xs) = x + suma xs
```

Si ejecutamos esta función para [1,2,3,4,5] obtenemos:

```
suma [1,2,3,4,5] -> 1 + (2 + (3 + (4 + (5 + 0) ) ) -> 15
```

Ejemplos de operaciones sobre listas

```
head :: [a] -> a
head (x:xs) = x
```

head, tail y last son funciones parciales, dan error para []

```
tail :: [a] -> [a]
tail (x:xs) = xs
```

```
last :: [a] -> a
last [x] = x
last (x:xs) = last xs
```

[x] es simplemente una forma abreviada para (x:[])

```
length :: [a] -> Int
length [] = 0
length (x:xs) = 1 + length xs
```

Todas estas funciones vienen predefinidas en el prelude de Haskell

Polimorfismo

Consideremos las siguientes funciones:

```
drop :: Int -> [a] -> [a]
```

```
drop n [] = []
```

```
drop 0 xs = xs
```

```
drop n (x:xs) = drop (n-1) xs
```

Descarta los primeros n
elementos de la lista

```
take :: Int -> [a] -> [a]
```

```
take 0 xs = []
```

```
take n [] = []
```

```
take n (x:xs) = x:take (n-1) xs
```

Toma los primeros n
elementos de una lista

En `[a]`, `a` puede ser cualquier tipo (es una variable de tipos).

Se dice que `drop` y `take` son funciones polimórficas.

Sinónimos de tipos

Podemos definir sinónimos de tipos con el constructor `type`:

```
type String = [Char]
```

Es la definición de String
en Haskell

También por ejemplo:

```
type Pos = (Int, Int)
```

Posiciones en un tablero

Y podemos usar este tipo nuevo:

```
type Board = [Pos]
```

Un tablero es una lista de
posiciones

Tipos definidos por el programador

Podemos definir tipos con nuevos valores:

```
data Bool = False | True
```

Y se pueden definir tipos inductivos

```
data Nat = Zero | Succ Nat
```

Definen los naturales por medio de dos constructores:

```
Zero :: Nat          y          Succ :: Nat -> Nat
```

Algunos valores del tipo son:

```
Zero, Succ Zero, Succ (Succ Zero), ...
```

Una función de ejemplo:

```
toInt :: Nat -> Int
toInt Zero = 0
toInt (Succ m) = 1 + toInt m
```

Tipos definidos por el programador

Podemos definir tipos con nuevos valores:

```
data Bool = False | True
```

Y se pueden definir tipos inductivos

```
data Nat = Zero | Succ Nat
```

Definen los naturales por medio de dos constructores:

```
Zero :: Nat          y          Succ :: Nat -> Nat
```

Algunos valores del tipo son:

```
Zero, Succ Zero, Succ (Succ Zero), ...
```

Una función de ejemplo:

Ejercicio: Definir la suma y el producto sobre Nat

```
concat (Succ m) = 1 + concat m
```


Conjuntos

Podemos implementar conjuntos usando tipos inductivos de la siguiente manera:

```
data Set a = Vacio | Ins a (Set a)
```

Algunos ejemplos de funciones sobre conjuntos:

```
insert :: (Eq a) => a -> Set a -> Set a
insert x Vacio = Ins x Vacio
insert y (Ins x xs) | x == y = (Ins x xs)
                    | x /= y = Ins x (insert y xs)
```

```
contains :: (Eq a) => a -> Set a -> Bool
contains _ Vacio = False
contains x (Ins y xs) = x == y || contains x xs
```


Conjuntos

Podemos implementar conjuntos usando tipos inductivos de la siguiente manera:

```
data Set a = Vacio | Ins a (Set a)
```

Algunos ejemplos de funciones sobre conjuntos:

```
insert :: (Eq a) => a -> Set a -> Set a
insert x Vacio = Ins x Vacio
insert y (Ins x xs) | x == y = (Ins x xs)
                   | x /= y = Ins x (insert y xs)
```

```
contains :: (Eq a) => a -> Set a -> Bool
```

```
contains _ Vacio = False
```

```
cont
```

Ejercicio: Definir la unión de conjuntos

Actividades

- Leer los capítulos 1-5 de "A Gentle Introduction to Haskell". P. Hudak, J. Peterson & J. Fasel. 1998.
Disponible en: <https://www.haskell.org/tutorial/>

Bibliografía

- "A gentle introduction to Haskell". P. Hudak, J. Peterson & J. Fasel. 1998. Disponible en: <https://www.haskell.org/tutorial/>